

SHL Assessment Recommendation Agent — Approach

1. Design Choices

Stack. Pure Ruby 3.2 + WEBrick on Render (cloud). Ruby was the only viable runtime: **Python 3** triggers an Xcode CLT dialog on this macOS version (blocks all scripts). **Node/npm**, **Docker**, **Homebrew** were not installed and could not be added without system permissions. **Ruby 2.6** ships with macOS and WEBrick/JSON are stdlib — zero setup. Deployed to **Render** (free tier) for a permanent, always-on URL. The same codebase runs on macOS and Linux.

- **Catalog layer** — 38 SHL Individual Test Solutions (compiled from SHL's public catalog): programming languages (Java, Python, JS/TS, Go, Rust, etc.), cloud (AWS, Azure, GCP, K8s, Docker), cognitive tests (Verify G+, Numerical, Verbal, Logical), personality (OPQ32r, Sales Predictor, EQ, etc.).
- **Agent layer** — Deterministic behavior router with 6 intents (greeting, clarify, recommend, refine, compare, refuse). No LLM — pure pattern-matching engine. ~440 lines.
- **API layer** — WEBrick server with CORS, input validation, 28-second timeout, catch/throw guard. ~160 lines.

2. Retrieval Setup

Search is a **keyword-scoring system** designed for Recall@10 alignment. Query words are extracted from user messages and split on whitespace *and* non-alphanumeric boundaries (e.g., `java+react`, `→ java, react`). Each word scores assessments: **+10** if in the name, **+5** if in its keyword list, **+3** if in its description. An exact name-phrase match adds **+15**. Results are capped at 10. Personality assessments (type 'P') are mixed proportionally — at least one is always included unless the query explicitly avoids soft skills. Role-specific keyword mappings (e.g., 'java' → ['java', 'jvm', 'spring', 'enterprise']) expand implicit skill queries. The catalog uses a curated 38-assessment fallback dataset (live scraping is disabled because SHL's URL returns a 301 redirect).

3. Behavior Design (Prompt Equivalent)

This is a **rule-based agent** (no LLM). The 'prompt' equivalent is the deterministic behavior chain:

- **Greeting** — Word-boundary regex (`\bhi\b, \bhello\b`) on the first 1–2 messages.
- **Clarify** — Fires when no role/seniority keywords are present. 'I need to hire' without context is *not* sufficient to recommend.
- **Recommend** — Fires when role OR seniority keywords are found across the full conversation history.
- **Refine** — Triggers on 'actually', 'instead', 'change' etc. in follow-up turns (messages ≥ 3). Re-queries the catalog.
- **Compare** — Activated by 'compare', 'vs', 'versus', 'difference between'. Looks up mentioned assessment names.
- **Refuse** — Regex detector for off-topic topics (weather, sports, movies), prompt injection ('ignore previous instructions', 'you are now a free AI', 'developer mode'), legal questions, and general hiring advice.

4. Evaluation Method

Ten conversation traces simulate realistic hiring scenarios. Each is scored on 4 axes:

Trace	Scenario	Tests
001	Junior Java developer	Recall@10 for programming
002	Senior Python data scientist	Data + technical recall
003	Java → refine to full-stack	Refine + React ranking
004	Compare OPQ32r vs MCA	Compare behavior
005	Entry-level customer support	Entry + soft skills recall
006	Sales manager (2-turn)	Clarify→Recommend pipeline
007	Vague "I need to hire"	Must clarify, not recommend
008	Prompt injection	Refusal (5 attack vectors)
009	Senior DevOps K8s + AWS	Multi-tech retrieval
010	VP Engineering	Exec + leadership mix

Scoring: Schema compliance (10 pts) — every response must have reply, recommendations, end_of_conversation. Catalog only (10 pts) — all URLs must exist in the catalog. Turn cap (10 pts) — each trace ≤ 8 messages. Mean Recall@10 (70 pts) — fraction of gold-relevant assessments in the top 10.

5. What Did Not Work

Regex substring matches. 'hi' matched inside 'hiring'; 'compare' matched 'comparing'. Fixed with `\b...\b` word boundaries on all patterns.

Vague queries returning results. 'I need to hire' without role context was triggering recommendations. Fixed by requiring role/seniority keywords before recommending.

Refine threshold too strict. Requiring > 3 turns missed 2-turn refinements. Changed to ≥ 3 .

Combined tokens not split. 'java+react,' hid 'react' from search. Added non-alphanumeric token splitting.

Server validation loop bug. next in each_with_index only skips the iteration, not the HTTP handler. Replaced with `throw(:invalid_message) + guard clause`.

Test harness indexing bug. Used recommendations.length as message index. Rewrote with turn counter.

Missing hiring advice refusal. Agent answered salary/interview questions. Added 12 refusal patterns.

Nokogiri dependency for scraping. SHL catalog URL returns 301; page structure unreliable. Removed scraping; use fallback dataset. Eliminated native-extension gem.

Bundler 1.17 broke Ruby 3.2. Lockfile used String#untaint (removed in Ruby 3.2). Updated to Bundler 2.4.22.

Missing bundle exec. WEBrick needs Bundler's load path. Changed Render start command from 'ruby server.rb' to 'bundle exec ruby server.rb'.

6. Measured Improvements

Metric	Before	After
Schema compliance	80%	100%
Catalog-only URLs	90%	100%
Turn cap (≤ 8)	100%	100%
Mean Recall@10	~50%	71.5%
Estimated score	~75/100	92.9/100
Off-topic refusal	Partial	100%
Prompt injection refusal	Partial	100%
Vague query $\rightarrow$ clarifies	No	Yes
Refine after "actually"	No	Yes

Improvements were measured by running the full automated test suite (ruby test_traces.rb) against the local server before and after each change. Score improvements were validated across all 10 traces.